

Web Scraping - get the Flight & Train info from Chrome

Using the UiPath Data Scraping box on the project's travel website (flights + trains)

What this does: Chrome really opens the travel page and UiPath really extracts the **Available Flights** table and the **Available Trains** table, then the project picks the cheapest of each and compares them. The site you scrape is **LiveDemoSite/index.html** (included in the project).

Why not MakeMyTrip? Real fare sites forbid scraping and use CAPTCHA. So we scrape our own travel page - the fares are labelled DEMO DATA, but the web scraping is 100% real and the same steps work on any site you are authorised to automate. (See MAKEMYTRIP_NOTICE.md.)

0. One-time setup

- Install **Google Chrome**.
- Install the UiPath Chrome extension: Studio → Home → Tools → UiPath Extensions → Chrome → Install (turn it On in Chrome).
- Manage Packages → make sure **UiPath.UIAutomation.Activities** is installed (it provides Data Scraping; already in project.json).

1. Open the travel site in Chrome

In your project folder open the **LiveDemoSite** folder and **double-click index.html**. Chrome shows an **Available Flights** table and an **Available Trains** table. Keep the tab open.

2. Scrape the FLIGHTS table

- 1 In Studio open **Workflows\SearchFlights.xaml** (the box labelled "SCRAPE THE FLIGHTS TABLE HERE").
- 2 Ribbon: **Design** → **Data Scraping**.
- 3 In Chrome, **click a cell in the Flights table** (e.g. IndiGo), then **click the next row** (Air India).
- 4 The preview shows all 6 columns already filled: **Airline, FlightNumber, DepartureTime, ArrivalTime, Duration, Fare** → **Next**. (If asked "Extract Correlated Data?" click **No** - a real table comes in one shot.)
- 5 Maximum number of results = **0** (all rows) → **Finish**.
- 6 Studio adds a browser scope + **Extract Table Data** and a DataTable variable.
- 7 Set the Extract Table Data **output** to **out_FlightsTable** (or add one Assign: out_FlightsTable = ExtractDataTable).

3. Scrape the TRAINS table (same steps)

- 1 Open **Workflows\SearchTrains.xaml** ("SCRAPE THE TRAINS TABLE HERE").
- 2 **Design** → **Data Scraping** → in Chrome scroll to the **Available Trains** table.
- 3 Click a cell (e.g. Tamil Nadu SF Express), then the next row (Grand Trunk Express).
- 4 Preview: TrainName, TrainNumber, DepartureTime, ArrivalTime, Duration, Fare → Next → Max results **0** → Finish.
- 5 Set the Extract Table Data **output** to **out_TrainsTable**.

4. Run it - the scrape now drives the comparison

- 1 Open **Main.xaml** → **Run**.
- 2 In the run-mode dialog type **Live** (not Demo). 'Live' tells the project to call the scrape workflows.
- 3 Chrome opens, both tables are scraped, and the **three result pop-ups** (cheapest flight, cheapest train, cheapest overall) are computed from the **scraped data**.

If a scrape returns nothing, the project **automatically falls back to the built-in sample data**, so the three outputs always appear.

5. (Optional) Prove the scrape worked

After the Extract Table Data activity in SearchFlights.xaml, add an **Input Dialog** (Label =):

```
"Scraped " + out_FlightsTable.Rows.Count.ToString + " flights. First: " +  
out_FlightsTable.Rows(0)("Airline").ToString + " Rs." + out_FlightsTable.Rows(0)("Fare").ToString
```

Use the **Run File** button on that workflow to see the scraped count on its own.

Troubleshooting

Data Scraping greyed out	Install the UiPath Chrome extension and keep the Chrome tab focused.
Only one column captured	Click a table cell (the text), not the heading; a real <table> brings all columns together.
Only 1-2 rows	Set Maximum results = 0 (all rows) in the wizard.
Column names differ	Rename to Airline/FlightNumber/DepartureTime/ArrivalTime/Duration/Fare (and TrainName/... for trains) so the Fare column is found.
Extract Table Data won't load	Reinstall UiPath.UIAutomation.Activities via Manage Packages, then reopen the file.
Blocked on a real fare site	That site forbids scraping - use this permitted page or another you are authorised to automate.